

FinRelief AI — Technical Architecture

SmartBridge Internship Program

Version 2.0.0

1. Architectural Approach

FinRelief AI is built as two fully decoupled services that only ever talk to each other over a REST API: a React single-page frontend and a FastAPI backend. The backend never renders HTML — every response is JSON — which keeps the two sides independently deployable and independently scalable. In production, the frontend runs on Vercel and the backend runs on Render, and either one can be redeployed or scaled without touching the other.

2. Component Map

Component	Description	Technology
User Interface	Responsive single-page application: login/register, dashboard, loan management, settlement simulation, letter generation	React 19 + Vite 8, Tailwind CSS v4
Charts	Stress trend and DTI visualisations on the dashboard	Recharts
Icons	UI iconography	Lucide React
API Client	Authenticated HTTP calls from frontend to backend	Axios
Application Backend	REST API: auth, loan CRUD, settlement calculation, letter generation, dashboard aggregation	FastAPI (Python)
ORM	Maps Python model classes to database tables	SQLAlchemy 2.0
Production Database	Persistent relational store for users, loans, settlements, and letters	Neon serverless PostgreSQL
Test Database	Isolated, disposable database used only during automated tests	SQLite
Authentication	Token issuance and verification	JWT (python-jose) + bcrypt password hashing

Component	Description	Technology
Rate Limiting	Protects API endpoints from abuse	SlowAPI
AI Integration	Drafts the OTS negotiation letter body	Google Gemini 2.0 Flash
PDF Generation	Renders the negotiation letter as a downloadable PDF	ReportLab
Frontend Linting	Enforces code quality on the client	oxlint
Test Framework	Backend automated test suite	Pytest + HTTPX

3. Request Lifecycle

The following describes what happens end-to-end for a typical feature — generating a negotiation letter:

1. The React frontend sends `POST /letters/{loan_id}` via Axios, with the JWT attached in the `Authorization` header.
2. FastAPI verifies the JWT and validates the request through Pydantic before any business logic runs.
3. The letters router pulls the loan's stored financial data and calls the Gemini client to draft the letter body.
4. If the Gemini API key is valid and the call succeeds, the letter is generated and tagged as AI-sourced.
5. If the Gemini call fails — timeout, non-2xx response, or an unparseable response — the backend automatically falls back to a static professional template instead, and the letter is tagged accordingly.
6. The letter is saved through SQLAlchemy into Neon PostgreSQL, and the JSON response is returned to the frontend for display or PDF download.

This same fallback pattern is what keeps the letter-generation feature functional even when the external AI service is unavailable — a small design decision, but one that matters more than most in a feature that borrowers may be relying on at a stressful moment.

4. Database Design

The schema is organised around four related entities:

- **Users** — account details, hashed passwords, and authentication metadata
- **Loans** — the borrower's loan details: lender, amount, EMI, overdue days, monthly income
- **Settlement Snapshots** — a record of each settlement calculation run against a loan, including DTI ratio and stress level, used to power the dashboard's trend charts over time
- **Negotiation Letters** — the generated letter body, along with whether it came from Gemini or the fallback template, and a version history per loan

Each loan can have multiple settlement snapshots and multiple letters over time, which is what allows the dashboard to show progress rather than a single static number.

5. Why Neon PostgreSQL Instead of a Local Database

The production database is Neon's serverless PostgreSQL rather than a locally hosted database file, for a reason specific to how the app is deployed. Render and Vercel both use ephemeral filesystems — anything written to local disk is wiped on every redeploy. A file-based database sitting in that environment would lose all user data the next time the app redeploys. Neon, by contrast, is hosted independently of the application's own deploy lifecycle, so the data persists regardless of how often the backend itself is redeployed.

SQLite is still used, but only for the isolated automated test suite, where a disposable, file-based database is actually the right tool — each test run gets a clean slate with no risk to real user data.

6. Application Characteristics

Scalability — The frontend and backend are deployed as separate services and can be scaled independently. Because the backend is stateless (JWT-based, no server-side session storage), multiple backend instances can run behind a load balancer without any shared session state to coordinate.

Security — Passwords are hashed with bcrypt before storage; sessions are handled through signed JWTs rather than server-side session storage; secrets (database URL, JWT signing key, Gemini API key) are loaded from environment variables and never committed to source control; SlowAPI rate-limits sensitive endpoints against abuse.

Availability — The Gemini-with-fallback pattern described above means the negotiation letter feature keeps working even during an AI service outage, which is the single most availability-relevant design decision in the system.

Performance — The settlement calculation itself is a pure, synchronous function with no external calls, so it returns near-instantly; the only latency-sensitive step in the system is the Gemini call during letter generation, which is time-boxed so a slow or failed AI response cannot hang the request indefinitely.